Implementation and Verification of the ChaCha20 Stream Cipher

Kumar Dipti Manna

July 30, 2026

1 Introduction

ChaCha20 is a stream cipher designed by Daniel J. Bernstein as a variant of the earlier Salsa20 cipher [1]. It is widely deployed in modern cryptographic protocols, including TLS 1.3 and the WireGuard VPN protocol, largely due to its high performance in software and resistance to timing side-channel attacks, since it avoids data-dependent branches and table lookups.

This implementation follows the IETF variant defined in RFC 8439 [2], which fixes the nonce at 96 bits and the block counter at 32 bits, distinguishing it from Bernstein's original specification that used a 64-bit counter and 64-bit nonce.

2 Algorithm Overview

ChaCha20 operates on a 4×4 matrix of 32-bit words (512 bits total), initialized as follows:

- **Words 0–3:** Fixed constants, the ASCII string `"expand 32-byte k"` encoded as four little-endian 32-bit words.
- **Words 4–11:** The 256-bit (32-byte) secret key.
- **Word 12:** A 32-bit block counter.
- **Words 13–15:** A 96-bit (12-byte) nonce.

2.1 The Quarter-Round Function

The core mixing primitive is the *quarter-round*, which operates on four 32-bit words (a, b, c, d) using a sequence of additions, XORs, and fixed left-rotations:

$$\begin{aligned} a \mathrel{+}= b; \quad d \mathrel{\oplus}= a; \quad d = \mathrm{ROTL}(d, 16) \\ c \mathrel{+}= d; \quad b \mathrel{\oplus}= c; \quad b = \mathrm{ROTL}(b, 12) \\ a \mathrel{+}= b; \quad d \mathrel{\oplus}= a; \quad d = \mathrm{ROTL}(d, 8) \\ c \mathrel{+}= d; \quad b \mathrel{\oplus}= c; \quad b = \mathrm{ROTL}(b, 7) \end{aligned}$$

This is implemented directly as shown in Listing 1.

Listing 1: The quarter-round operation.

```c
static void quarter_round(uint32_t *a, uint32_t *b, uint32_t *c, uint32_t *d)
{
    *a += *b; *d ^= *a; *d = ROTL32(*d, 16);
    *c += *d; *b ^= *c; *b = ROTL32(*b, 12);
    *a += *b; *d ^= *a; *d = ROTL32(*d, 8);
    *c += *d; *b ^= *c; *b = ROTL32(*b, 7);
}
```

2.2 The Block Function

A single ChaCha20 block is generated in `chacha20_block()` through the following steps:

1. Initialize the 16-word state as described above.
2. Copy the state into a working buffer, since the original state must be preserved for the final feed-forward step.
3. Perform 20 rounds, organized as 10 *double-rounds*. Each double-round applies four quarter-rounds to the matrix *columns*, followed by four quarter-rounds to the matrix *diagonals*. This alternating pattern ensures rapid diffusion of every input bit across the entire 512-bit state.
4. Add the original (pre-round) state back into the working state, word-by-word. This feed-forward step is essential: without it, the block function would be a simple invertible permutation rather than a one-way pseudorandom function, and the cipher would be trivially breakable.
5. Serialize the resulting 16 words to 64 bytes in little-endian order, producing the keystream block.

2.3 Encryption via XOR

Because ChaCha20 is a stream cipher, encryption and decryption are the same operation: the keystream is XORed with the input. For a message longer than 64 bytes, `chacha20_xor()` generates successive keystream blocks, incrementing the block counter after each one, and XORs 64-byte chunks (or a shorter final chunk) against the input buffer, as shown in Listing 2.

Listing 2: Stream encryption via repeated block generation and XOR.

```c
void chacha20_xor(const uint8_t key[32], const uint8_t nonce[12],
                  uint32_t counter, const uint8_t *in, uint8_t *out,
                  size_t len)
{
    uint8_t keystream[64];
    while (len > 0) {
        chacha20_block(key, counter, nonce, keystream);
        counter++;
        size_t block_len = (len < 64) ? len : 64;
        for (size_t i = 0; i < block_len; i++)
            out[i] = in[i] ^ keystream[i];
        in += block_len; out += block_len; len -= block_len;
    }
}
```

Since XOR is self-inverse, calling `chacha20_xor()` a second time with the same key, nonce, and initial counter on the ciphertext recovers the original plaintext.

3 Correctness Verification

Correctness was verified using the official test vector from RFC 8439, Section 2.4.2 [2]. This vector specifies a known 256-bit key, a 96-bit nonce, an initial block counter of 1, and a plaintext string (the well-known "sunscreen speech" excerpt), along with the expected ciphertext.

The verification procedure was:

1. Encrypt the known plaintext using the specified key, nonce, and counter.
2. Compare the resulting ciphertext (printed as hexadecimal) against the RFC's published value.
3. Decrypt the ciphertext using the same parameters and confirm the recovered plaintext exactly matches the original input.

3.1 Result

The implementation's output ciphertext,

```
6e2e359a2568f98041ba0728dd0d6981e97e7aec1d4360c20a27afccfd9fae0bf91b65c5524733ab8f593dabcd62b3571639d624e6515
500d6a6156a38e088a22b65e52bc514d16ccf806818ce91ab77937365af90bbf74a35be6b40b8eedf2785e42874d
```

matches the RFC 8439 reference value exactly, and decryption correctly recovered the original plaintext. This confirms the implementation is spec-compliant for at least this test case. No additional test vectors (e.g., all-zero key/nonce vectors, or multi-block vectors from RFC 8439 Appendix A) were exercised in this implementation; a more rigorous validation would include those as well.

4 Benchmarking Methodology

Two independent benchmarks were implemented to characterize performance from different angles: a cycle-accurate measurement of the core permutation, and a wall-clock throughput measurement of the full encryption path.

4.1 Portability Considerations

Because the code targets both Windows and POSIX systems, and both x86 and non-x86 architectures, the benchmarking code uses conditional compilation (`#if defined(...)`) to select the correct APIs at compile time:

- **Cycle counting** uses the `__rdtsc()` intrinsic from `x86intrin.h`, available only on x86/x64. This section is guarded by `#if defined(__x86_64__) || defined(__i386__)` so the code still compiles (with a fallback message) on ARM or other architectures.
- **Wall-clock timing** uses `QueryPerformanceCounter` on Windows (`_WIN32`) and `clock_gettime(CLOCK_MONO` `...)` on POSIX systems, wrapped in a single portable helper function, `wall_time_seconds()`.

4.2 Block-Level Cycle Counting

The first benchmark isolates the cost of `chacha20_block()` alone, excluding the XOR loop and any buffer management overhead. The procedure:

1. A warm-up loop of 1,000 calls runs first, allowing the CPU's branch predictor, instruction/-data caches, and dynamic clock frequency (e.g., Turbo Boost) to reach a steady state before the timed region begins.
2. The Time Stamp Counter is read via `__rdtsc()` immediately before and after a loop of 100,000 calls to `chacha20_block()`.
3. The difference in counter values gives total elapsed CPU cycles, from which cycles-per-block and cycles-per-byte (dividing by 64, the block size) are derived.

4.3 Wall-Clock Throughput

The second benchmark measures realistic encryption throughput using `chacha20_xor()` over a 1 MiB buffer, repeated 200 times (200 MiB total). A single warm-up call precedes the timed loop. Elapsed wall-clock time is measured with `wall_time_seconds()`, and throughput is reported in MiB/s.

This measurement necessarily includes overhead absent from the block-level test: the XOR loop itself, pointer arithmetic, and memory traffic to and from the input/output buffers. As a result, the wall-clock-derived cycles/byte figure is consistently somewhat higher than the block-level figure, which is expected and consistent across all test runs.

5 Conclusion

The implemented ChaCha20 cipher was verified against the official RFC 8439 test vector and produced the exact expected ciphertext, with correct round-trip decryption. Two complementary benchmarking approaches were designed and implemented to characterize performance: a cycle-accurate measurement of the core permutation via `rdtsc`, and a wall-clock throughput measurement of the full encryption path. This dual approach was designed to isolate the cost of the core algorithm from overhead introduced by the surrounding XOR loop and memory operations, providing a more complete performance picture than a single measurement method alone.

References

[1] D. J. Bernstein, *ChaCha, a variant of Salsa20*, 2008. https://cr.yp.to/chacha/chacha-20080128.pdf

[2] Y. Nir and A. Langley, *ChaCha20 and Poly1305 for IETF Protocols*, RFC 8439, IETF, 2018. https://www.rfc-editor.org/info/rfc8439/

[3] Anthropic, *Claude* (AI language model), 2026. https://claude.ai